

Akhil Sathish Kumar

Presenter Pack

Member B — Maze Generation, Difficulty Balance & Predictive Enemy

Slides 1 & 5 + live demo

Maze Runner · Problem Statement 15

REVA University · Hackathon अभिनव · 07/10/2026

SECTION 1 — The project (everyone must know this)

What we built

A maze game where the player has to reach the exit while an enemy chases them using real pathfinding. The enemy recalculates the shortest route to the player on every single step, using either BFS or A*, and you can switch between the two live and watch the difference on screen.

From level 4 a **second enemy** joins running the *other* algorithm, so BFS and A* hunt the same player at the same time.

How it works, in 60 seconds

The maze is a grid of 1s and 0s — 1 is a wall, 0 is floor. The player and each enemy sit on one cell. Sixty times a second we read the keyboard, move everything one cell if its cooldown has expired, and redraw. The enemy decides where to go by running a search from itself to the player, then stepping onto the second cell of the route it found.

Everything else is detail on top of that.

The five facts everyone must be able to say

1. **BFS uses a FIFO queue, so it finds the shortest path.** It finishes every cell at distance d before touching anything at $d+1$, so the first time it reaches the goal it got there in the fewest steps.
2. **A* scores cells** $f = g + h$ — g is steps taken, h is Manhattan distance remaining. h must never overestimate or optimality is lost.
3. **A* expanded ~26% fewer cells but ran slower** in wall-clock time, because its heap costs $O(\log n)$ per operation against the deque's $O(1)$.
4. **The enemy used to camp the exit** — "farthest from the player" is the opposite corner, which is where the exit is. Fixed by maximising `min(distance to player, distance to exit)`.
5. **We tuned difficulty by measurement, not feel** — `autoplay.py` ran 500 bot playthroughs and found the enemy was only $1.1\times$ slower than the player.

The numbers

Tech	Python 3 + pygame-ce. No dataset, no network, no hardware
Tests	226 headless checks, all passing
Benchmark	50 mazes per configuration
Balance	500 bot playthroughs; L1–3 ~68–70%, L4–10 20–38%
Repo	github.com/Hyperval/maze-runner

Who built what

Person	Owns
Akhil Sathish Kumar	Maze generation, braiding, difficulty balance
Akshay N	Pathfinding — BFS, A*, distance maps
Abdullah Subhaan K	Rendering, menu, pause, HUD, test suite
Christen Mendis	benchmark.py, autoplay.py, docs, demo video

SECTION 2 — Your code: `maze.py` and the balance tuning

You own how the maze is built and how hard the game is. Two files: `maze.py`, and the tuning constants in `settings.py`.

How the maze is generated — recursive backtracker

1. Start at cell (1,1), mark it floor.
2. Look at neighbours TWO cells away that are still solid wall.
3. Pick one at random. Knock out the wall BETWEEN them. Move there.
4. No unvisited neighbours? Pop the stack and back up.
5. Done when the stack is empty.

It is depth-first search used to **carve** rather than to search.

The two questions you will be asked

"Why two cells away?"

So walls always land on even coordinates and stay one cell thick. If we carved one cell at a time the whole grid would dissolve into one open room with no walls at all.

"Why must COLS and ROWS be odd?"

The carver only ever stands on odd coordinates. An even width would leave a ragged half-carved column at the edge.

Braiding — your most interesting decision

A freshly carved maze is "**perfect**": exactly one route between any two cells. That sounds good but means it is nothing but dead ends — the player gets cornered with zero counterplay.

`braid()` finds dead ends (cells with exactly one open side) and knocks out one extra wall with 35% probability, creating loops and therefore escape routes.

It had a second effect we measured. Loops also widen A*'s advantage over BFS, from ~11% to ~26%, because the search finally has real alternatives to discriminate between. A tight maze constrains the corridors so much that the heuristic has nothing to choose between.

That is a genuinely good thing to say out loud — it shows a design decision that turned out to have a measurable second consequence you didn't plan for.

Your demo piece: `visualise_maze.py`

```
python visualise_maze.py
```

Watch the carver tunnel forward, hit a dead end, back up its stack, and tunnel off in a new direction. **SPACE** pauses, **R** gives a new maze, **UP/DOWN** changes speed, **B** toggles braiding so you can see the extra walls it opens in green.

This is the single best thing you can show. If an examiner asks how the maze is generated, don't describe it — run this, pause it mid-carve, and point.

Screenshots: `assets/demo_carving.png` (mid-carve), `assets/demo_carved.png` (finished, with braided walls in green).

`carve_steps()` and the `yield` keyword

```
def carve_steps(...):
    ...
    yield grid          # hand the grid back, then PAUSE here
```

A function containing `yield` is a **generator**. It doesn't run to completion — each `yield` hands a value to whoever is looping over it and freezes there until the next iteration asks for more. That's what lets us draw the maze being carved one step at a time without storing hundreds of copies.

Your second feature: the predictive enemy (`PredictiveEnemy` in `entities.py`)

An ordinary enemy paths to the cell the player is standing on, so by the time it arrives they have moved. It is permanently one step behind and can only win by being faster. Yours asks a different question: **where will they be, and can I get there first?**

Stage 1 — `predict_route()`. Walk forward from the player along their heading. Keep going while the way ahead is open. When it is blocked, look at the other exits: exactly one way on is a forced corridor bend, so follow it, because the player has no other option either. Two or more is a junction — **stop**. We cannot know which way they will turn and guessing is worse than not guessing.

Stage 2 — `choose_target()`. The player reaches `route[i]` after roughly `i × player_delay` milliseconds. We reach it after `our_distance × our_delay`. Walking the route backwards from the far end, take the **deepest** cell where we arrive no later than they do — deepest because a cut-off further along is harder to escape. Nothing qualifies? Interception is off, chase normally.

The measurements that matter

Interception rate within 20 cells	23–32%
Beyond ~30 cells	Essentially never — the lookahead bounds the commitment
Cost per step	~721 cells vs BFS's 328, about 2.2×
As a third enemy	Broke the game: L6–10 fell to 6–14% win rate
As a replacement for enemy 2	L6–10 hold at 22–38%

Two bugs you found building it

The freeze. When the enemy was already standing on its own intercept cell, the path had length 1, `update()` returned early without moving, and because the repath counter had just been reset it never re-planned — frozen for the rest of the level. Both `Enemy` and `PredictiveEnemy` now force a replan on the next tick. The base class had the same latent bug, it just almost never triggered.

The dishonest HUD. It reported only the path search, so the predictive enemy showed 8 cells against BFS's 401 while quietly running a whole-maze flood fill. It now reports the total. Worth volunteering if anyone asks about cost — "we caught ourselves under-reporting it" lands well.

The balance constants you tuned

Constant	Value	Effect
<code>PLAYER_MOVE_DELAY</code>	90 ms	How often the player can step
<code>ENEMY_MOVE_DELAY</code>	190 ms	Starting enemy speed
<code>ENEMY_SPEEDUP_PER_LEVEL</code>	11 ms	How fast difficulty ramps
<code>ENEMY_MIN_DELAY</code>	140 ms	Floor — the fastest the enemy ever gets
<code>ENEMY_HEAD_START_MS</code>	2200 ms	Enemies hold still at level start
<code>BRAID_CHANCE</code>	0.35	Fraction of dead ends opened into loops

The single most important relationship: `ENEMY_MIN_DELAY` (140) must stay **above** `PLAYER_MOVE_DELAY` (90). If the enemy's delay drops below the player's, the enemy moves more often than you do, and since it always walks a shortest path it catches you with mathematical certainty. Skill stops mattering.

We originally had it at 100ms — only $1.1\times$ slower than the player — and the bot tester found the late levels were effectively unwinnable.

Your likely viva questions

- "How is the maze generated?" — recursive backtracker, above.
- "What is braiding and why?" — perfect maze = all dead ends = no counterplay.
- "How did you pick the difficulty numbers?" — measured with `autoplay.py`, tuned until L1–3 sat around 70% and later levels around 20–40%.
- "What happens if the enemy is faster than the player?" — it catches you with certainty, which is why the floor is above the player's delay.

SECTION 3 — Your presenting job

You present **Slide 1 (Cover)** and **Slide 5 (Future Scope)**. You open the presentation and you close it. Keep both short — the middle three slides and the live demo are where the marks are.

Slide 1 — Cover (target: 20 seconds)

Say roughly this:

Good morning. We're presenting problem statement 15 — a maze game where the enemy chases the player using pathfinding. I'm Akhil, and this is Akshay, Abdullah and Christen.

Then hand to Akshay. **Do not** read out the three cards on the slide; the examiners can see them. The temptation is to explain everything on the cover — don't, you'll run out of time for the demo.

If they ask what you personally built, one line: *"I built the maze generation and tuned the difficulty."*

Slide 5 — "An enemy that cuts you off" (target: 60 seconds)

This slide is your feature. It used to be a future-scope list; you built the headline item, so the slide now shows it working. Take the full minute.

Open with the contrast:

The other two enemies path to where the player *is*, so they always trail — they can only win by being faster. This one asks where you're going to *be*.

Stage 1, and this is the interesting part:

It walks forward from the player along the direction they're moving, following corridor bends where there's only one way on. At a junction it *stops* — it genuinely can't know which way you'll turn, and guessing there is worse than not guessing. So the prediction is honest about where its knowledge runs out.

Stage 2:

Then one flood fill gives it the walking distance to every cell, and it takes the deepest cell on your predicted route that it can reach no later than you. That's the yellow box on screen. If it can't beat you anywhere, interception is off and it just chases.

The trade-off — say this before they ask:

It isn't free. It costs about 721 cells of thinking per step against BFS's 328, roughly 2.2 times as much, because of the extra flood fill.

Then close:

That's our project. The code and documentation are on GitHub. Happy to take questions.

Don't trail off. Finish on a clear sentence and stop talking.

Follow-ups you will get on this slide

- "*How often does it actually intercept?*" — 23–32% of the time when it's within 20 cells of the player, and essentially never beyond about 30, because the lookahead bounds how far ahead it can commit.
- "*Why not look further ahead?*" — we raised it from 14 to 22 cells after measuring that at 14 the parameter was the binding limit 51% of the time rather than the maze's junctions. Past that, routes end at junctions anyway.
- "*Is it just a better enemy?*" — no, and we measured that too. Added as a *third* enemy it broke the game, levels 6–10 fell to 6–14% win rate. It replaces the second enemy instead, so the threat count stays at two and the progression is "the second enemy gets smarter", not "more enemies".
- "*Does it ever get it wrong?*" — constantly, and that's the point. Every time the player turns off the predicted route the intercept is wasted. That's what it pays for the ability with.

During the rest of the presentation

- **You answer anything about the maze or difficulty**, even on someone else's slide. If an examiner asks Christen "why is level 1 easier than level 5", that's yours — step in.
- You also own the question "*is the game actually winnable?*" — the answer is the 500-run measurement, L1–3 around 68–70%.

If something goes wrong

- Demo crashes → Christen plays the backup video. Don't apologise repeatedly; one sentence, move on.
- You blank on a question → "*Akshay owns that part, he can give you the detail.*" Handing over cleanly is fine. Silence is not.

Final checklist

- [] Read Section 1 of this document until the five facts are automatic
- [] Practise your 20-second cover opener out loud, twice
- [] Know which ONE future-scope item you'll talk about
- [] Have `maze.py` open in a tab in case they ask to see the carving code